

Design Project (CS F376)

A REPORT ON

**Development of a large scale generalized model for multi-organ
nuclei segmentation**

SUBMITTED BY

Student Name: Nishal Ahmed P. K

ID NO: 2023A7PS0209U



**BITS Pilani, Dubai Campus
Dubai International Academic City, Dubai
UAE**

SUBMITTED TO

Dr. Tojo Mathew

(SEPTEMBER 2025 –OCTOBER 2025)

ABSTRACT

Accurate instance segmentation of nuclei is a task which has applications in cell classification, disease diagnosis, drug development, and treatment. The automation of such a task through digital pathology can aid in alleviating the rising demand for medical health professionals. Several methods ranging from classical image processing to complex machine learning algorithms have been used to solve this problem, and methods based on convolutional neural networks (CNN) give the best results. Newer methods of using pretrained LLMS and attention-based models have been proposed. However, most of these methods show unsatisfactory results when tested against unseen datasets, which makes them unreliable for large-scale deployment. This report will explore the existing approaches to nuclei instance segmentation, reimplement and evaluate them. This report will focus on developing a Mask RCNN based pipeline which leverages vague regions and shows better AJI, PQ and Dice scores on the NuInsSeg dataset when benchmarked against a plain Mask RCNN pipeline.

All the code pertaining to this project is available at:

<https://github.com/nishalahmedpk/NuInsSeg>

LIST OF FIGURES

Figure 1: NuInsSeg sample data for three human organs.	2
Figure 2: Architectural diagram of a classical U-Net used for nuclei instance segmentation [1].	5
Figure 3: Testing the trained model on four different human organs with visualization using matplotlib.	6
Figure 4: The dual decoder model for nuclei instance segmentation in [4].	7
Figure 5: Testing the trained model on a random human organ with visualization using matplotlib.	8
Figure 6: The Mask R-CNN framework [8].	9
Figure 7: MLAA Net Architecture [2].	10
Figure 8: Detailed Architectural Diagram of HoVerNet [9].	11
Figure 9: HoVerNet Pipeline [9].	11
Figure 10: Network Structure of CellViT	12
Figure 11: Manually annotated vague masks (rightmost) in NuInsSeg, where even annotaters found difficulty marking nuclei.	13
Figure 12: Segmentation result of a sample with high vague area. Vague down weighted model (Top) and Base model (Bottom)	14

LIST OF TABLES

Table 1: Performance of reimplementatation of shallow UNet on the NuInsSeg dataset compare with the results shown in [7].	6
Table 2: Performance of reimplementatation of dual decoder UNet on the NuInsSeg dataset compare with the results shown in [4] and the results before post processing.	8
Table 3: Dice, AJI and PQ after fine tuning MaskRCNN for the NuInsSeg dataset.	9
Table 4: Dice, AJI and PQ of base Mask RCNN compared with Reweighted MaskRCNN...	14

LIST OF LISTINGS

Listing 1: Code for initializing the all the layers of the UNet in Tensorflow [7].	17
Listing 2: Loss function definition and model compilation code.	18
Listing 3: Loss function definitions for the dual decoder model as provide in [4]......	19
Listing 4: Code for initializing the all the layers of the dual decoder UNet in Tensorflow. ...	19
Listing 5: Code to pretrained MaskRCNN model	23
Listing 6: Definition of function to convert instance to boxes for MaskRCNN.....	23
Listing 7: Function definition to set all seeds to ensure fair comparisons between runs.....	24

TABLE OF CONTENTS

ABSTRACT	ii
LIST OF FIGURES	iii
LIST OF TABLES	iv
LIST OF LISTINGS	v
TABLE OF CONTENTS	vi
Chapter 1: Introduction	1
1.1 Objectives	1
1.2 Report Structure	1
Chapter 2: Problem Definition and Dataset Exploration	2
2.1 Nuclei Instance Segmentation.....	2
2.2 NuInsSeg.....	2
Chapter 3: Traditional Image Processing and Evaluation Methods.....	3
3.1 Traditional Image Processing Methods	3
3.1.1 Watershed Method	3
3.1.2 Thresholding	3
3.2 Evaluation Method.....	3
3.2.1 Intersection over Union.....	3
3.2.2 Dice Coefficient	3
3.2.3 Aggregated Jaccard Index	3
3.2.4 Panoptic Quality.....	4
Chapter 4: Models, Experiments and Results.....	5
4.1 UNet.....	5
4.1.1 Implementation	5
4.1.2 Results and Insights	5
4.2 Dual Decoder UNet.....	7
4.2.1 Implementation	7
4.2.2 Results and Insights	8
4.3 Mask RCNN.....	8
4.3.1 Model Setup and Training	9
4.3.2 Results and Insights	9
4.4 Other Models	10
4.4.1 MLAA Net.....	10
4.4.2 HoVerNet.....	10
4.4.3 CellViT	11
Chapter 5: Nuclei Instance Segmentation with Mask R-CNN considering Vague Regions	13

5.1 Motivation and Novelty	13
5.2 Custom Pipeline.....	13
5.4 Results and Analysis	13
5.5 Future Scope	14
Chapter 6: Conclusion	15
5.1 Future Directions	15
References.....	16
Appendix A: Code Listings	17

Chapter 1: Introduction

The automation of accurate instance segmentation through digital pathology can aid in alleviating the rising demand for medical health professionals. This task has applications in cell classification, disease diagnosis, drug development, and treatment. Several methods ranging from classical image processing to complex machine learning algorithms have been used to solve this problem, and methods based on convolutional neural networks (CNN) give the best results [1]. Newer methods of utilizing pretrained LLMS and attention-based models have been proposed [1], [2], [3]. However, most of these methods show unsatisfactory results when tested against unseen datasets mainly due to domain shift caused by staining variability, which makes them unreliable for large-scale deployment.

1.1 Objectives

The primary objective of this project is to develop a deployable large scale generalized model for multi-organ nuclei segmentation on the NuInsSeg dataset. This project involves implementing, evaluating and performing ablation studies on existing models which display relatively high benchmark scores for nuclei instance segmentation [2], [4],[5]. Further ablation studies were done on pre and post-processing strategies as it has been proven to be one of the most essential components in yielding a high segmentation score in terms of Intersection Over Union, Dice Coefficient, Aggregated Jaccard Index (AJI) and Panoptic Quality (PQ) [6] and the result of such experiments is Chapter 5.

1.2 Report Structure

The report consists of 5 chapters starting off with chapter 1 discussing the objectives of project. It is followed by chapter 2 which provides an in-depth description of the problem and the dataset used for all required purposes in this project. Chapter 3 discusses traditional image processing methods, its role in complex machine learning pipelines. It also discusses evaluation methods used for segmentation and the intuition behind them. Chapter 4 provides a detailed overview of relevant models. Implemented pipelines will also contain results and insights from implementation. Chapter 5 explores a semi novel pipeline which leverages vague areas provided in the NuInsSeg dataset and yield better metrics compare to baseline. Chapter 6 provides a concise summary and conclusion.

Chapter 2: Problem Definition and Dataset Exploration

This chapter describes the need for a generalized model for nuclei instance segmentation and explore NuInsSeg, the base dataset used for training, testing and evaluating different methods.

2.1 Nuclei Instance Segmentation

Ever since the conceptualization of thinking machines by Alan Turing there have been efforts to automate various fields, and the medical imaging field was no exception. The field gained immense popularity with the advent brightfield and fluorescent digital scanners that produce and store whole slide images (WSIs) in digital form [7]. Within this field the segmentation of nuclei instances has many use cases. Cancer diagnosis can be done by exploiting characteristics specific to nuclei such nucleus-cytoplasm ratio or nuclei density. It can also be used to evaluate treatment quickly or to gain new insights about nuclei morphology [1].

2.2 NuInsSeg

NuInsSeg is one of the biggest manually annotated datasets of nuclei in Hematoxylin and Eosin (H&E) stained images, it contains 665 image patches with more than 30,000 segmented nuclei from 31 human and mouse organs [7]. For maintaining accuracy and reliability all the segmentation was done manually. Along with label and binary masks, they also provide ambiguous area masks and distance maps.

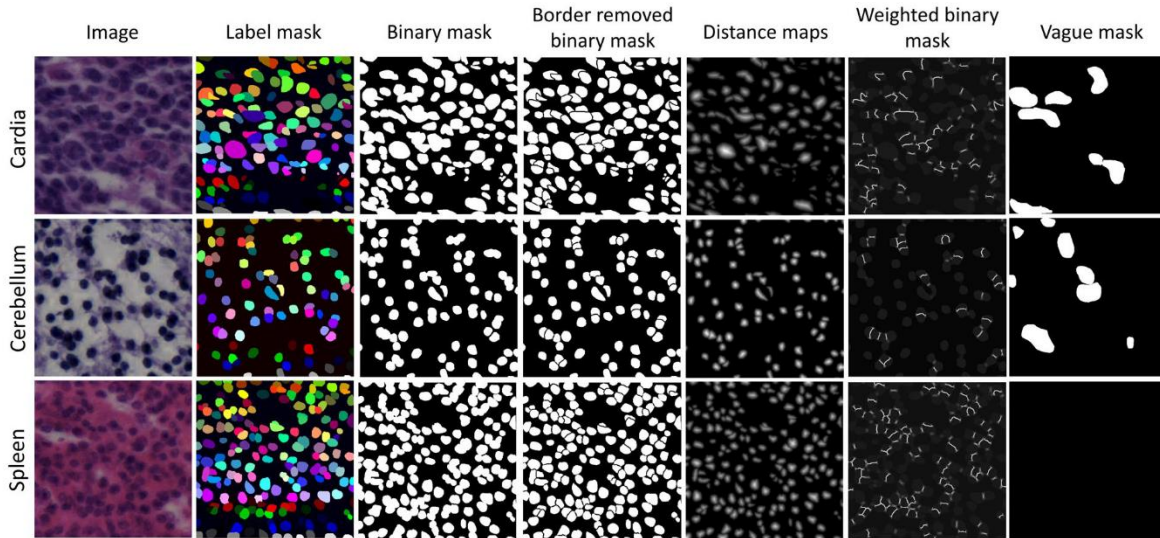


Figure 1: NuInsSeg sample data for three human organs.

Chapter 3: Traditional Image Processing and Evaluation Methods

This chapter discusses traditional image processing methods such as watershed, thresholding, Active Contour Model (ACM) and explain how they are still being used in modern machine learning pipelines. It also explains the evaluation methodologies used to benchmark and provide an accuracy score to segmentation results.

3.1 Traditional Image Processing Methods

Traditional methods use classical image processing and computer vision strategies. It alone cannot be used to implement a complete pipeline, however many of the techniques for pre and postprocessing. In this section we will discuss some of them.

3.1.1 Watershed Method

Watershed algorithm is widely used postprocessing method and is extremely popular for overlapping problems, it begins by detecting local minima and maxima to detect object regions [1]. The object is detected by growing regions with local maxima akin to the filling of water into a watershed and when two regions overlap, they are segmented.

3.1.2 Thresholding

Thresholding is method to divide an image based on intensity thresholds. It is computationally efficient but it fails when there are variations in stains and intensity which is often the case in nuclei segmentation [1].

There are several other traditional methods such as Active Contour Model (ACM) and clustering, however the experiments in this report does not make use of it as of now therefore we choose not to include it.

3.2 Evaluation Method

3.2.1 Intersection over Union

Intersection over Union (IoU) is a metric used to measure overlap between two objects, in this case nuclei.

$$IoU = \frac{X \cap Y}{X \cup Y} = \frac{TP}{TP + FP + FN}$$

3.2.2 Dice Coefficient

Dice Coefficient is used to measure the overlap between ground truth and the predicted segmentation.

$$Dice = \frac{2 \times (X \cap Y)}{|X| + |Y|} = \frac{2TP}{2TP + FP + FN}$$

3.2.3 Aggregated Jaccard Index

AJI is metric which measures similarity between the predicted segmentation results and the ground truth annotations [1].

$$AJI = \frac{\sum_{i=1}^n |x_i \cap y_i|}{\sum_{i=1}^n |x_i \cup y_i| + \sum_{f \in U} |y_f|}$$

In the equation i refers to an instance and the bottom right term represents the no unmatched predicted segments.

3.2.4 Panoptic Quality

An evaluation metric created to evaluate panoptic quality, also widely used in instance segmentation.

$$PQ = \frac{2 \times TP}{2 \times TP + FP + FN} \times \frac{\sum_{(x_i, y_i) \in TP} \text{IoU}(x_i, y_i)}{|TP|}$$

In the equation i refers to an instance.

There are several other methods used to evaluate instance segmentation, since this report only makes use of the aforementioned methods. Therefore, we do not include it in the report.

Chapter 4: Models, Experiments and Results

This chapter explores popular existing models and provides a detailed discussion on the reimplementation of the model if there exists one.

4.1 UNet

UNet provides a basis of multitude for a multitude of other complex pipelines. It employs a bottom-up approach with a symmetric encoder decoder architecture with simplistic skip connections which results in lightweight models with fast convergence.

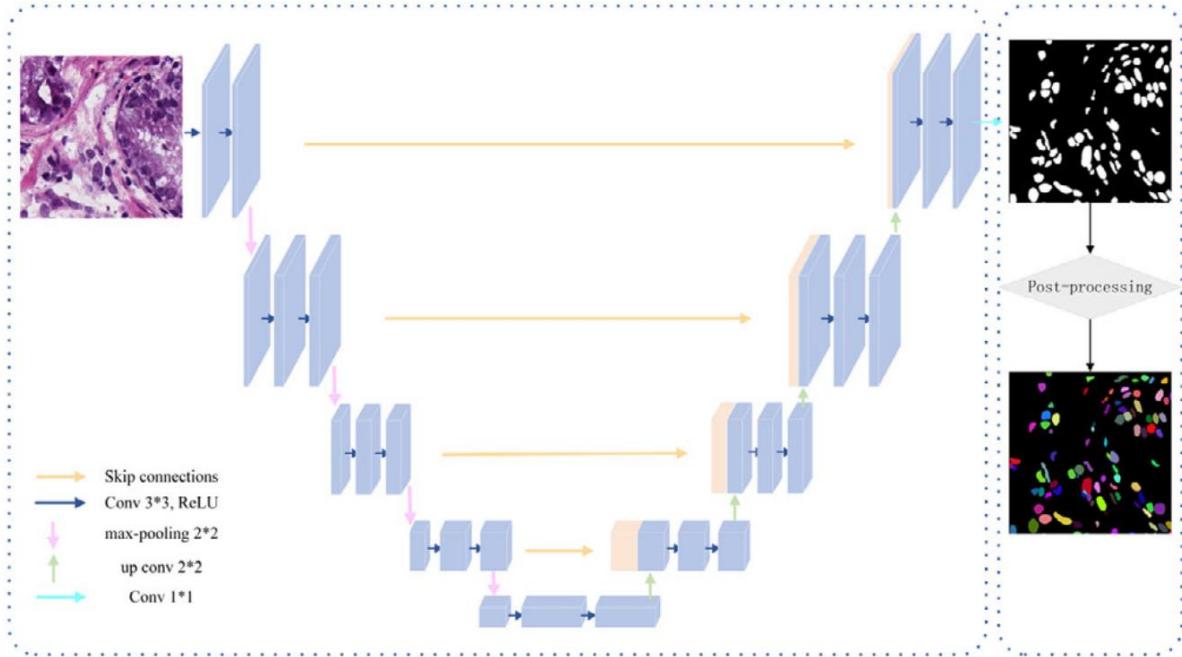


Figure 2: Architectural diagram of a classical U-Net used for nuclei instance segmentation [1].

4.1.1 Implementation

The reimplementation of the architecture and the training of the model was done exactly as recommended in [7]. This implementation has a depth of four levels with each level consisting of 2 convolutional layers with a dropout layer of rate 0.1 in between for regularization. The first level has a filter of size 16 and it is increased to 256 using max pooling layers and then brought back to 16 using transposed convolution. The final convolutional layer is passed into 1x1 convolution which has a sigmoid activation function.

This model is trained with an engineered loss function from Binary Cross Entropy (BCE) loss and Dice Loss and evaluated using the Dice Coefficient. The model was trained for 100 epochs dropping the learning rate by 0.5 after every 20 epochs, the model used Adams optimizer with an initial learning rate of 0.001 all as prescribed in the code given along with [7].

4.1.2 Results and Insights

The reimplementation of the shallow UNet yielded similar results to what was shown in [7], and the results are given in Table 1.

Metric (Average)	Results (%)	
	Reimplementation	Actual
Dice	78.0767136	78.8
Dice Watershed	78.0780558	--
Dice Watershed Wo Vague	78.9750036	--
Aggregated Jaccard Index (AJI)	40.5876498	50.5
AJI Watershed	46.5974438	--
AJI Watershed Wo Vague	48.0170044	--
Panoptic Quality (PQ)	39.7928562	42.7
PQ Watershed	37.2359624	--
PQ Watershed Wo Vague	38.4814588	--

Table 1: Performance of reimplementation of shallow UNet on the NuInsSeg dataset compare with the results shown in [7].

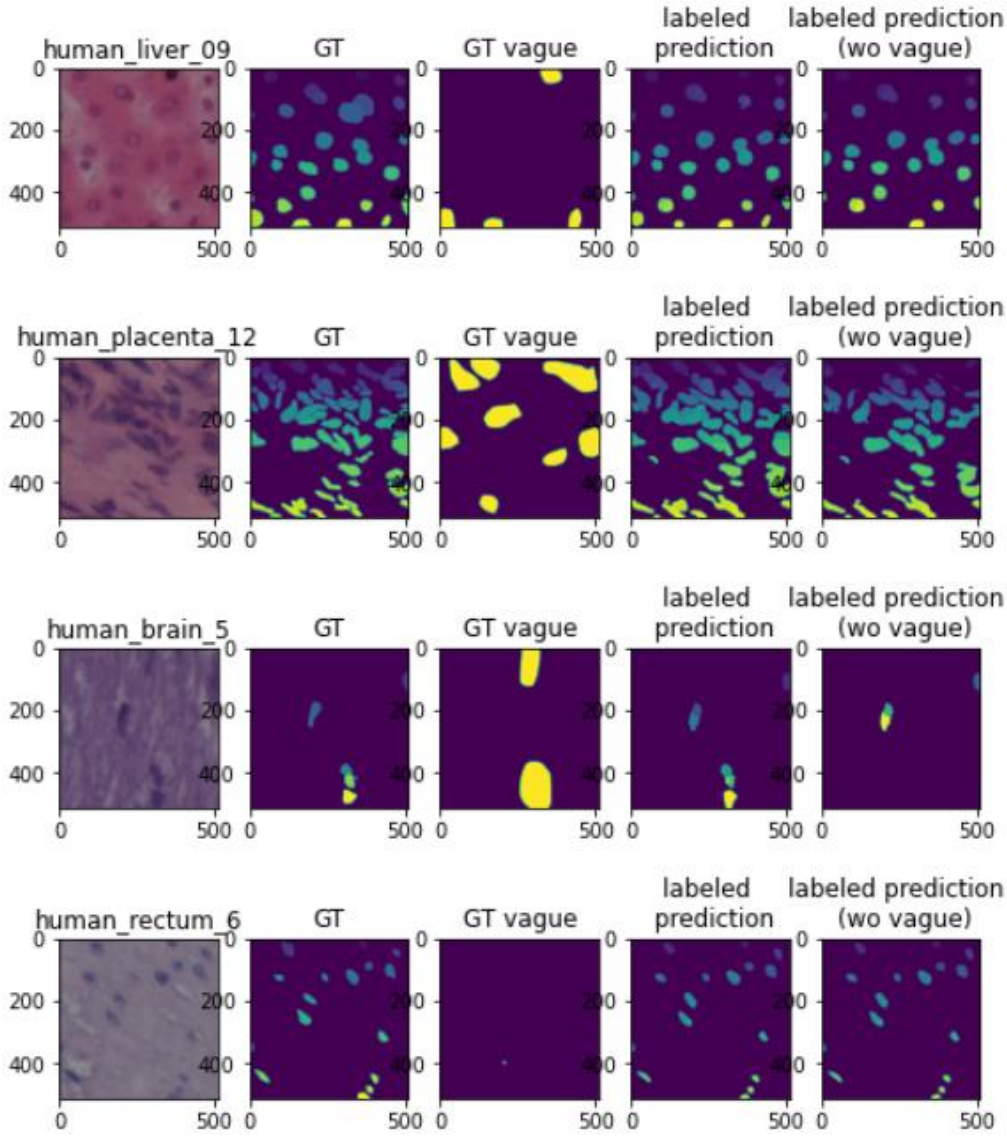


Figure 3: Testing the trained model on four different human organs with visualization using matplotlib.

The yielded results also reinforce confidence in the ability of the UNet architecture in its ability to be solid base model to other complex architectures. The results also show the

running watershed algorithms as a post processing tool results in marginal increase in AJI scores whereas and have negligible to decremental impact on PQ and Dice Scores. The decremental impact can be explained by over segmentation by the post-processing methods (the watershed algorithm) which introduces extra fragments that are absent in the actual image.

4.2 Dual Decoder UNet

This architecture which was first implemented for nuclei instance segmentation in [4] was said to provide the best results by the paper introducing the NuInsSeg dataset [7], therefore it was reimplemented. This architecture is built on a traditional UNet but instead of having one decoder branch like in explained in section 4.1 it has two decoder paths, one for predicting the binary mask and another for the distance mask. The final instance segmentation results are obtained by passing the predicted results into post processing step which will be explained in section 4.2.1.

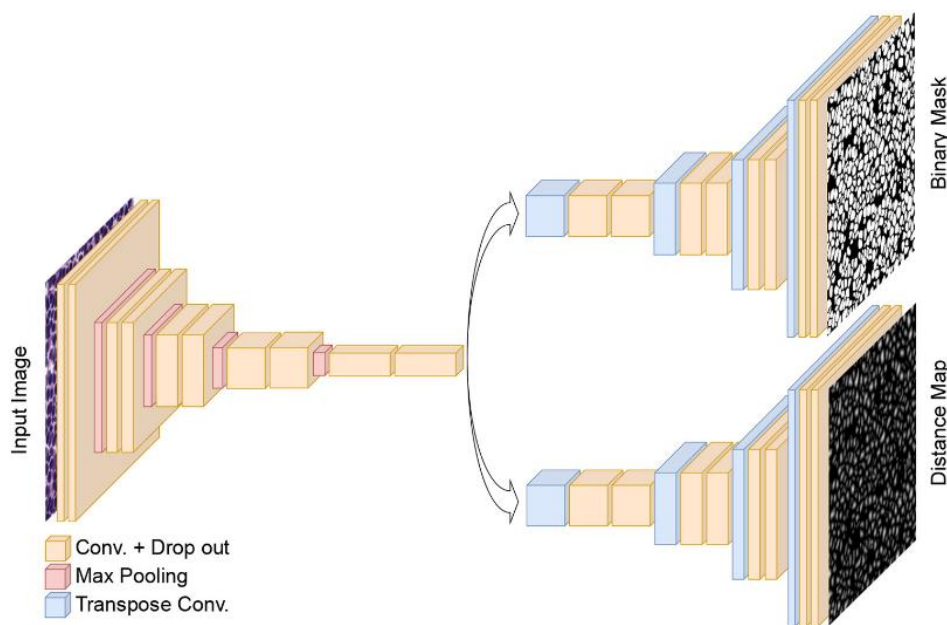


Figure 4: The dual decoder model for nuclei instance segmentation in [4].

4.2.1 Implementation

The reimplementaion of this architecture was not done exactly as shown in [4] as of the time of writing this report. The augmentation and the generation of additional ground truth mask for training the model mentioned in [4] was not implemented at the time of writing the report. The model contains five convolutional layers, the loss function for the decoder predicting the distance maps means squared error (MSE) and the loss function for the other decoder is a combination of Dice loss and binary cross entropy loss. The model and the loss function definition are as given listings 4 and 3 respectively.

The model was trained for 120 epochs dropping the learning rate by 0.1 after every 20 epochs, the model used Adams optimizer with an initial learning rate of 0.001 all as prescribed in [4].

4.2.2 Results and Insights

The reimplementation of the dual decoder UNet yielded results far different from what was shown in [4], the results before the post processing step which was the highlight of this pipeline ended up decreasing the existing scores.

Metric (Average)	Results (%)		
	Base UNet Head	Reimplementation	Actual
Dice	68.89	63.014972	79.4
Aggregated Jaccard Index (AJI)	32.55	31.046785	55.9
Panoptic Quality (PQ)	29.091411	25.184311	51.3

Table 2: Performance of reimplementation of dual decoder UNet on the NuInsSeg dataset compare with the results shown in [4] and the results before post processing.

The yielded results highlight the importance of preprocessing steps such as augmentation and generation of additional ground truth masks for the pipeline to generate expected results. It is also to noted that although this novel method provides better results when compared to others, it does so by increasing computational overhead. Further attempts at reimplementation were not pursued as the author denied access to a reference.

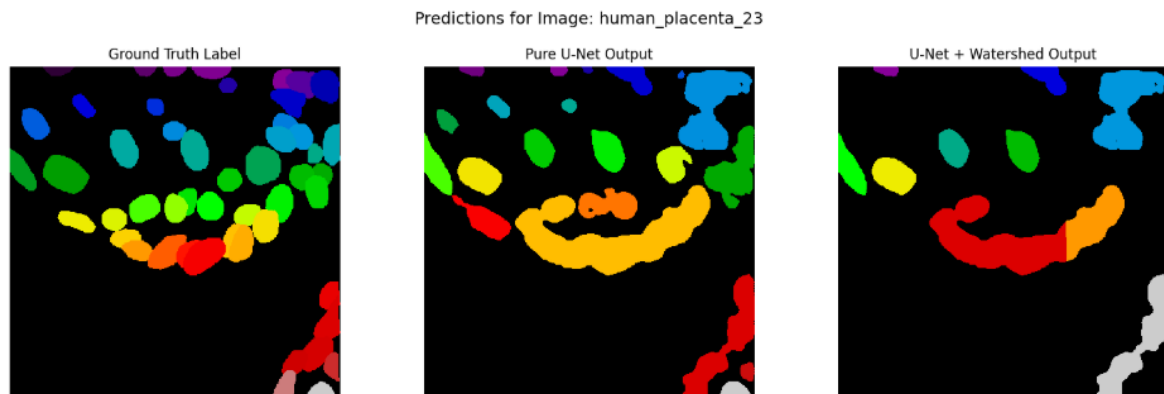


Figure 5: Testing the trained model on a random human organ with visualization using matplotlib.

4.3 Mask RCNN

Mask RCNN is top-down two stage approach to nuclei segmentation [1], the first stage consists of object detection also known as Region Proposal Network (RPN) which generates candidate regions where objects may exist, this is followed by layer which refines the bounding, performs classification [8] and provides an instance segmentation.

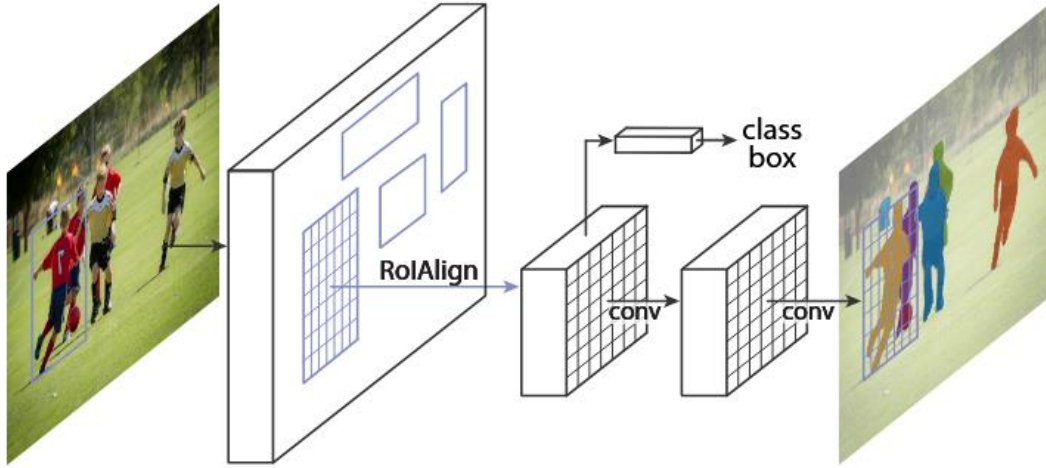


Figure 6: The Mask R-CNN framework [8].

4.3.1 Model Setup and Training

Since several implementations of Mask CNN exist in the form of libraries, a complete reimplementaion was not done. A pretrained model was loaded from the torchvision library and tuned to the NuInsSeg dataset.

In order to train the dataset on Mask RCNN, each individual nuclei should have a bounding box and binary mask associated with it. This can be derived from the instance labels as showing in Listing 6.

The pretrained model was trained for varying epochs (10,15,50) using a Stochastic Gradient Descent (SGD) with an initial learning rate of 0.005, momentum of .9, and weight decay of 0.0005 on the dataset. The learning rate was set to drop by a factor of 10 every 5 epochs.

4.3.2 Results and Insights

The Dice, AJI and PQ score obtained from the runs shows promise than the results obtained from the previous models, however it hard to make a fair comparison as runs discussed in 4.1 and 4.2 used K-Fold validation over 2 folds however Mask RCNN run used 20-80 test train split provided by the scikit library.

Metric	Results (%)		
	10 Epochs	15 Epochs	50 Epochs
Dice	79.21	81.04	79.77
Aggregated Jaccard Index (AJI)	57.94	59.95	58.11
Panoptic Quality (PQ)	58.85	60.95	58.86

Table 3: Dice, AJI and PQ after fine tuning MaskRCNN for the NuInsSeg dataset.

The model peaked at around 10-15 epochs then likely started overfitting, a more robust evaluation method using the K-Fold method suggested [7] should be applied in order to analyze the model more efficiently. It should also be noted that all the runs did not have the same random seed to begin with however traces of dice decreasing in the higher epoch ranges could be seen when analyzing the 50-epoch run. This is an expected result as the model we used is already pretrained on the COCO dataset. Since this model provided the highest Dice,

AJI, PQ scores thus far and due to its ease of implementation and modifications we will be leveraging to perform further studies such as ablation.

4.4 Other Models

This section contains pipelines/models which are not yet implemented but are in the scope of the project.

4.4.1 MLAA Net

This architecture is a heavily modified alteration of UNet which makes use of various intermediary pipelines which make use of attention layers. Instead of a regular convolutions blocks this architecture makes use Parallel Depthwise Convolution (PWC) module in the encoder and decoder, the regular max-pooling operation is replaced with a hybrid approach coined Hybrid Pooling in which max-pooling is combined with wavelet pooling to preserve more details [2]. The bottleneck is replaced by complex pipeline consisting convolutional, normalization, avg-pooling and max-pooling layers and is termed by the authors as Multi-scale Feature Refinement and Activation (MFRA) module, the skip connections are replaced by taking encoder and decoder and passing it in through pipeline which performs joint attention operations in order for multi-level feature aggregation [2]. [2] also reported better Dice and IoU scores for the NuInsSeg dataset, however these two scores are not enough to evaluate an automated nuclei segmentation pipeline. It is also to be noted that addition of complex pipelines and obtaining only marginal improvements over UNet cannot be an optimal solution due to the computational overhead tradeoff.

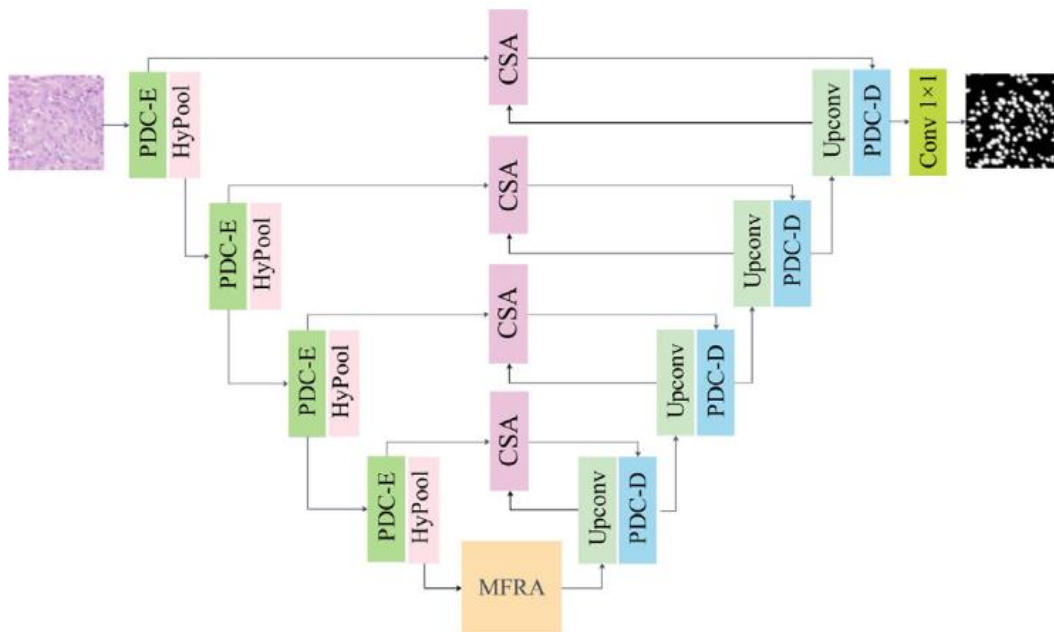


Figure 7: MLAA Net Architecture [2]

4.4.2 HoVerNet

HoVerNet has three branches of convolutional networks which predicts binary nuclei masks, horizontal and vertical distance maps, and nuclei classes respectively. For the instance segmentation problem the classification branch can be ignored.

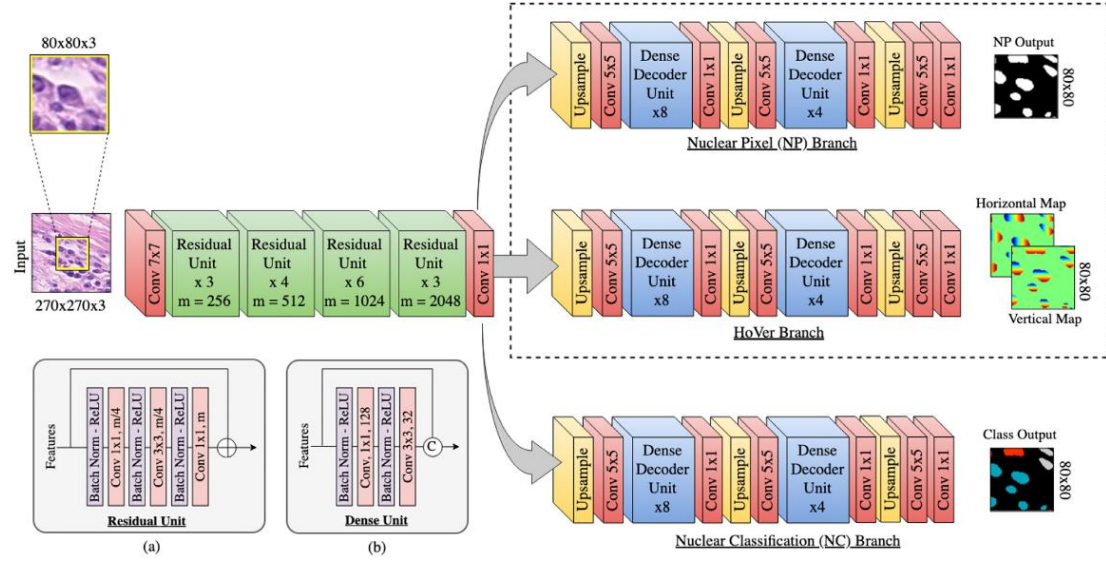


Figure 8: Detailed Architectural Diagram of HoVerNet [9]

The resulting pipeline looks similar to the DDUNet discussed in section 4.2 in the manner it predicts the binary masks and distance masks (Horizontal and Vertical separately) and proceeds to pass them on to a series of post-processing steps to obtain the final instance segmentation.

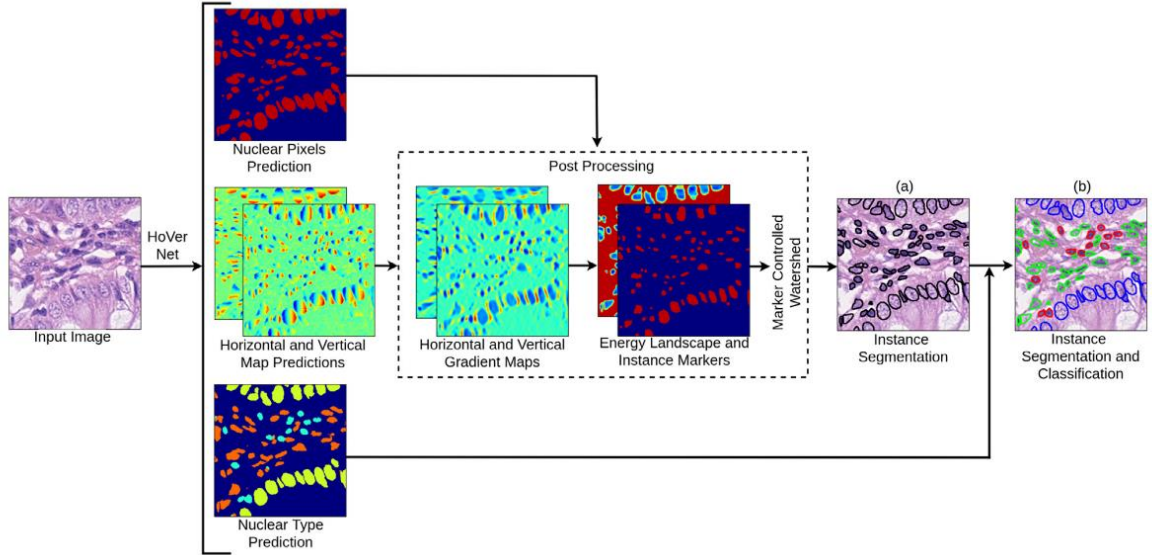


Figure 9: HoVerNet Pipeline [9]

Unlike DDUNet, HoVerNet claims to achieve state-of-the-art performance on complex datasets such as MoNuSeg, CoNSeP and CMP-17.

4.4.3 CellViT

CellViT has an encoder-decoder architecture similar to HoVerNet but it replaces CNN's with vision transformers [10]. It tokenizes each instance using pretrained Vision Transformer (ViT) encoders and decodes it into binary masks, distance maps and nuclei classification as done in HoVerNet using decoder network connected with skip connections [10]. It shows

great promise in nuclei instance segmentation but was not reimplemented due to time constraints.

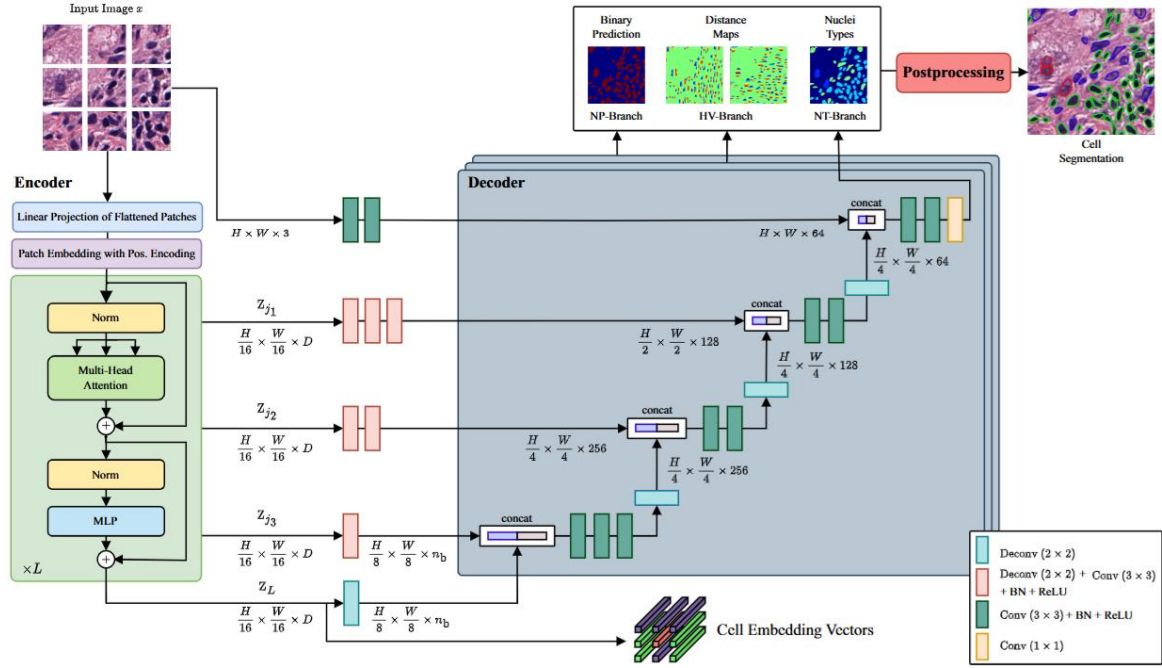


Figure 10: Network Structure of CellViT

Chapter 5: Nuclei Instance Segmentation with Mask R-CNN considering Vague Regions

This chapter introduces a semi-novel pipeline which leverages vague masks which were provided in [7] which provides marginal improvements in AJI and PQ scores and slight decrease in Dice score possibly due to noise compared to the baseline Mask RCNN discussed in section 4.3. It explains the motivation behind the development of the pipeline, details its development and training, and highlights results, inference and future scopes.

5.1 Motivation and Novelty

The NuInsSeg dataset provides manually annotated vague region along with label masks [7] and to the best of our knowledge most of prior work ignores them or treats them in pre/post-processing [4], [7].

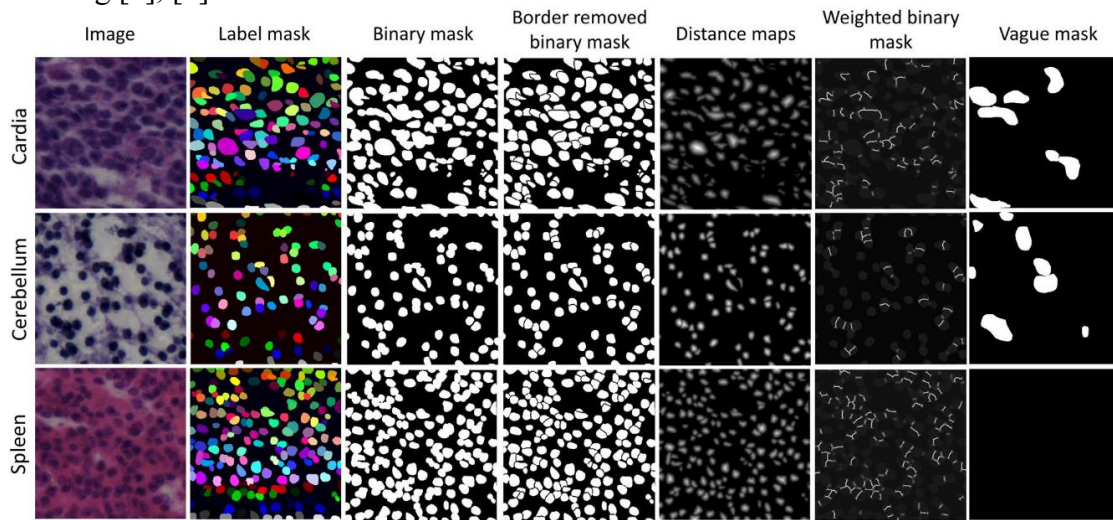


Figure 11: Manually annotated vague masks (rightmost) in NuInsSeg, where even annotators found difficulty marking nuclei.

Loss reweighting has shown to improve model performance especially when data is small and imbalanced, which is exactly the case here. Therefore, we modify Mask RCNN mask loss such that if the mask overlaps with a vague area we impose a small penalty on the original loss.

5.2 Custom Pipeline

The base model was set up exactly as done in section 4.3 and the same random seed was used for the base and the custom pipeline to ensure fair results. The base pipeline and loss reweighted pipeline were both trained for varying epochs 15 using a Stochastic Gradient Descent (SGD) with an initial learning rate of 0.005, momentum of .9, and weight decay of 0.0005 on the dataset. The learning rate was set to drop by a factor of 10 every 5 epochs.

5.4 Results and Analysis

The vague aware pipeline shows marginal improvements in AJI and PQ scores and slight decrease in Dice score possibly due to noise compared to the baseline Mask RCNN discussed in section 4.3.

Metric	Results (%) for 15 Epochs
--------	---------------------------

	Baseline	Vague Reweighted Mask RCNN
Dice	81.04	80.82
Aggregated Jaccard Index (AJI)	59.95	59.91
Panoptic Quality (PQ)	60.95	61.01

Table 4: Dice, AJI and PQ of base Mask RCNN compared with Reweighted MaskRCNN.

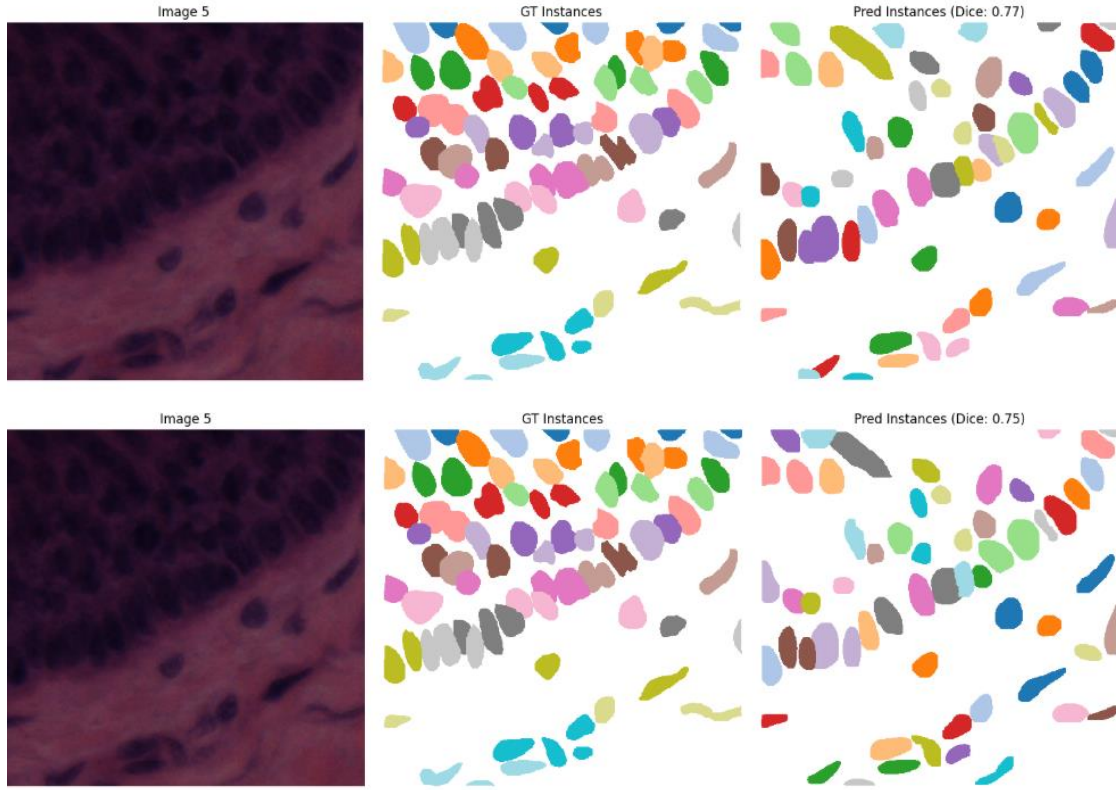


Figure 12: Segmentation result of a sample with high vague area. Vague down weighted model (Top) and Base model (Bottom)

Although the results of a fair evaluation are favors baseline in Dice marginal improvement can be seen in AJI and PQ which implies down weighting vague regions can result in slight improvement of extremely meaningful metrics such as AJI and PQ. However these results do not capture the entire picture of the models performance and a KFold test train split as mentioned in [7] should be implemented in a fair environment. Further works which will be discussed in section 5.5

5.5 Future Scope

To the best of our knowledge there exists no nuclei instance segmentation models which leverages vague masks and has potential for publication. The custom pipeline discusses in section 5.2 only experiments down weighting by a factor of 0.3, meaningful results can be obtained by using different factors and curriculum learning. Future work may also include pipelines with a custom head to predict vague regions so that the model may be fine tuned specific domains. Similar strategies can be applied to other models to yield results better than baseline.

Chapter 6: Conclusion

This chapter concludes the report and explores insights, discusses results and explores future directions of the project.

Nuclei instance segmentation is a problem with high impact in the medical imaging domain, however methods and pipelines used to address it often fall short when it comes to generalizing to different types of data which is how data is often processed in the real world. Novel methods exist but they often obtain similar results to tried and tested methods such UNet's or provide only marginal improvements for specific edge cases. However, the existence of literature with different novel approaches is essential to guide future researchers to prepare to tackle the challenge. The failure of DDUNet implementation discussed in Chapter 4 highlights and reinforces the importance carefully tuned image processing steps in nuclei instance segmentation. The custom pipeline introduced in Chapter 5 forms a promising basis for publication.

5.1 Future Directions

Attention based models seem to show superior results at certain edge cases, reimplementing more complex pipelines will aid in understanding the results in the edge case. CellViT shows promises of capturing more semantic detail while segmenting. Performing ablation studies on pre-and-post-postprocessing can help streamline or develop a completely new pipeline which may or may not show better results. Pre and post processing steps which leverage the manually annotated vague areas in the NuInsSeg remain unexplored. Quantum image processing seems to show promise with companies such as IBM promising a fault tolerant quantum system by 2029. However, a fully operational quantum pipeline for nuclei segmentation seems unlikely to appear due to current hardware limitations.

References

- [1] Y. Chen, Q. Huang, M. Geng, Z. Wang, and Y. Han, “A Systematic Review on Cell Nucleus Instance Segmentation,” *IET Image Process.*, vol. 19, no. 1, p. e70129, 2025, doi: 10.1049/ipr2.70129.
- [2] S. Zhu, Y. Li, L. Lu, T. Mao, Y. Yan, and X. Dai, “MLAANet: A multi-level attention-aware network for pathological nuclei segmentation,” *Biomed. Signal Process. Control*, vol. 112, p. 108716, Feb. 2026, doi: 10.1016/j.bspc.2025.108716.
- [3] J. Chen *et al.*, “TransUNet: Rethinking the U-Net architecture design for medical image segmentation through the lens of transformers,” *Med. Image Anal.*, vol. 97, p. 103280, Oct. 2024, doi: 10.1016/j.media.2024.103280.
- [4] A. Mahbod, G. Schaefer, G. Dorffner, S. Hatamikia, R. Ecker, and I. Ellinger, “A dual decoder U-Net-based model for nuclei instance segmentation in hematoxylin and eosin-stained histological images,” *Front. Med.*, vol. 9, Nov. 2022, doi: 10.3389/fmed.2022.978146.
- [5] J. Chen, G. Yang, A. Liu, X. Chen, and J. Liu, “SFE-Net: Spatial-Frequency Enhancement Network for robust nuclei segmentation in histopathology images,” *Comput. Biol. Med.*, vol. 171, p. 108131, Mar. 2024, doi: 10.1016/j.compbiomed.2024.108131.
- [6] A. Mahbod, G. Dorffner, I. Ellinger, R. Woitek, and S. Hatamikia, “Improving generalization capability of deep learning-based nuclei instance segmentation by non-deterministic train time and deterministic test time stain normalization,” *Comput. Struct. Biotechnol. J.*, vol. 23, pp. 669–678, Jan. 2024, doi: 10.1016/j.csbj.2023.12.042.
- [7] A. Mahbod *et al.*, “NuInsSeg: A fully annotated dataset for nuclei instance segmentation in H&E-stained histological images,” *Sci. Data*, vol. 11, no. 1, p. 295, Mar. 2024, doi: 10.1038/s41597-024-03117-2.
- [8] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” Jan. 24, 2018, *arXiv*: arXiv:1703.06870. doi: 10.48550/arXiv.1703.06870.
- [9] S. Graham *et al.*, “Hover-Net: Simultaneous segmentation and classification of nuclei in multi-tissue histology images,” *Med. Image Anal.*, vol. 58, p. 101563, Dec. 2019, doi: 10.1016/j.media.2019.101563.
- [10] F. H $\ddot{o}$ rst *et al.*, “CellViT: Vision Transformers for Precise Cell Segmentation and Classification,” Oct. 06, 2023, *arXiv*: arXiv:2306.15350. doi: 10.48550/arXiv.2306.15350.

Appendix A: Code Listings

Listing 1: Code for initializing the all the layers of the UNet in Tensorflow [7].

```
c1 = Conv2D(16, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (inputs)

c1 = Dropout(0.1) (c1)

c1 = Conv2D(16, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c1)

p1 = MaxPooling2D((2, 2)) (c1)


c2 = Conv2D(32, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (p1)

c2 = Dropout(0.1) (c2)

c2 = Conv2D(32, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c2)

p2 = MaxPooling2D((2, 2)) (c2)


c3 = Conv2D(64, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (p2)

c3 = Dropout(0.1) (c3)

c3 = Conv2D(64, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c3)

p3 = MaxPooling2D((2, 2)) (c3)


c4 = Conv2D(128, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (p3)

c4 = Dropout(0.1) (c4)

c4 = Conv2D(128, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c4)

p4 = MaxPooling2D(pool_size=(2, 2)) (c4)


c5 = Conv2D(256, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (p4)

c5 = Dropout(0.1) (c5)

c5 = Conv2D(256, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c5)
```



```

u6 = Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same') (c5)
u6 = concatenate([u6, c4])

c6 = Conv2D(128, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (u6)

c6 = Dropout(0.1) (c6)

c6 = Conv2D(128, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c6)


u7 = Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same') (c6)
u7 = concatenate([u7, c3])

c7 = Conv2D(64, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (u7)

c7 = Dropout(0.1) (c7)

c7 = Conv2D(64, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c7)


u8 = Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same') (c7)
u8 = concatenate([u8, c2])

c8 = Conv2D(32, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (u8)

c8 = Dropout(0.1) (c8)

c8 = Conv2D(32, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c8)


u9 = Conv2DTranspose(16, (2, 2), strides=(2, 2), padding='same') (c8)
u9 = concatenate([u9, c1], axis=3)

c9 = Conv2D(16, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (u9)

c9 = Dropout(0.1) (c9)

c9 = Conv2D(16, (3, 3), activation='relu',
kernel_initializer='glorot_uniform', padding='same') (c9)


outputs = Conv2D(1, (1, 1), activation='sigmoid') (c9) # for binary

```

Listing 2: Loss function definition and model compilation code.

```
def dice_coef(y_true, y_pred):
```

```

smooth = 1.

y_true_f = K.flatten(y_true)
y_pred_f = K.flatten(y_pred)

intersection = K.sum(y_true_f * y_pred_f)

return (2. * intersection + smooth) / (K.sum(y_true_f) +
K.sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

def bce_dice_loss(y_true, y_pred):
    return 0.5 * keras.losses.binary_crossentropy(y_true, y_pred) -
dice_coef(y_true, y_pred)

model = Model(inputs=[inputs], outputs=[outputs])

model.compile(optimizer = adam_v2.Adam(learning_rate= LearnRate), loss=
bce_dice_loss , metrics=[dice_coef])

```

Listing 3: Loss function definitions for the dual decoder model as provide in [4].

```

def dice_coef(y_true, y_pred):
    smooth = 1.

    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)

    intersection = K.sum(y_true_f * y_pred_f)

    return (2. * intersection + smooth) / (K.sum(y_true_f) +
K.sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

def bce_dice_loss(y_true, y_pred):
    return 0.5 * tensorflow.keras.losses.binary_crossentropy(y_true,
y_pred) - dice_coef(y_true, y_pred)

def mse_score(y_true, y_pred):
    return K.mean(K.square(y_pred - y_true), axis=-1)

```

Listing 4: Code for initializing the all the layers of the dual decoder UNet in Tensorflow.

```

s = Lambda(lambda x: x / 255)(inputs)

```

```

c1 = Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(s)

c1 = Dropout(0.1)(c1)

c1 = Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c1)

p1 = MaxPooling2D((2, 2))(c1)


c2 = Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(p1)

c2 = Dropout(0.1)(c2)

c2 = Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c2)

p2 = MaxPooling2D((2, 2))(c2)


c3 = Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(p2)

c3 = Dropout(0.2)(c3)

c3 = Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c3)

p3 = MaxPooling2D((2, 2))(c3)


c4 = Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(p3)

c4 = Dropout(0.2)(c4)

c4 = Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c4)

p4 = MaxPooling2D(pool_size=(2, 2))(c4)


c5 = Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(p4)

c5 = Dropout(0.1)(c5)

c5 = Conv2D(512, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c5)


## decoder for dis unet

u6 = Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c5)

```

```

u6 = concatenate([u6, c4])

c6 = Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(u6)

c6 = Dropout(0.2)(c6)

c6 = Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c6)


u7 = Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c6)
u7 = concatenate([u7, c3])

c7 = Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(u7)

c7 = Dropout(0.2)(c7)

c7 = Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c7)


u8 = Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same')(c7)
u8 = concatenate([u8, c2])

c8 = Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(u8)

c8 = Dropout(0.1)(c8)

c8 = Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c8)


u9 = Conv2DTranspose(16, (2, 2), strides=(2, 2), padding='same')(c8)
u9 = concatenate([u9, c1], axis=3)

c9 = Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(u9)

c9 = Dropout(0.1)(c9)

c9 = Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal',
padding='same')(c9)


outputs1 = Conv2D(1, (1, 1), activation='linear', name='output_dis')(c9)


## decoder for segmentation unet

u6_seg = Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c5)

```

```

u6_seg = concatenate([u6_seg, c4])

c6_seg = Conv2D(128, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(u6_seg)

c6_seg = Dropout(0.2)(c6_seg)

c6_seg = Conv2D(128, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(c6_seg)


u7_seg = Conv2DTranspose(64, (2, 2), strides=(2, 2),
padding='same')(c6_seg)

u7_seg = concatenate([u7_seg, c3])

c7_seg = Conv2D(64, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(u7_seg)

c7_seg = Dropout(0.2)(c7_seg)

c7_seg = Conv2D(64, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(c7_seg)


u8_seg = Conv2DTranspose(32, (2, 2), strides=(2, 2),
padding='same')(c7_seg)

u8_seg = concatenate([u8_seg, c2])

c8_seg = Conv2D(32, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(u8_seg)

c8_seg = Dropout(0.1)(c8_seg)

c8_seg = Conv2D(32, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(c8_seg)


u9_seg = Conv2DTranspose(16, (2, 2), strides=(2, 2),
padding='same')(c8_seg)

u9_seg = concatenate([u9_seg, c1], axis=3)

c9_seg = Conv2D(16, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(u9_seg)

c9_seg = Dropout(0.1)(c9_seg)

c9_seg = Conv2D(16, (3, 3), activation='relu',
kernel_initializer='he_normal', padding='same')(c9_seg)


outputs2 = Conv2D(1, (1, 1), activation='sigmoid',
name='output_seg')(c9_seg)

```

```

model_dual_path = models.Model(inputs=[inputs], outputs=[outputs1,
outputs2])

model_dual_path.compile(optimizer=Adam(learning_rate=LearnRate),

                        loss={'output_dis': 'mean_squared_error',
'output_seg': bce_dice_loss},

                        loss_weights= {'output_dis': 1.0, 'output_seg':
1.0},

                        metrics={'output_seg':dice_coef,
'output_dis':mse_score}))

```

Listing 5: Code to pretrained MaskRCNN model

```

import torchvision

from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor

def get_maskrcnn_model(num_classes=2): # bg & nuclei
    model = torchvision.models.detection.maskrcnn_resnet50_fpn(
        weights="DEFAULT"
    )

    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)

    in_features_mask =
model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    model.roi_heads.mask_predictor = MaskRCNNPredictor(
        in_features_mask, hidden_layer, num_classes
    )

    return model

```

Listing 6: Definition of function to convert instance to boxes for MaskRCNN.

```

def instance_labels_to_masks_and_boxes(label_img, min_area=25):

```

```

instance_ids = np.unique(label_img)
instance_ids = instance_ids[instance_ids != 0]

masks_list = []
boxes_list = []

for inst_id in instance_ids:
    mask = (label_img == inst_id)
    area = mask.sum()
    if area < min_area:
        continue
    coords = np.where(mask)
    if len(coords[0]) == 0:
        continue
    y_min, y_max = coords[0].min(), coords[0].max()
    x_min, x_max = coords[1].min(), coords[1].max()

    if x_max > x_min and y_max > y_min:
        boxes_list.append([x_min, y_min, x_max, y_max])
        masks_list.append(mask)

if len(masks_list) == 0:
    return np.zeros((0, 4)), np.array([]), np.zeros((0,
*label_img.shape), dtype=bool)

masks = np.stack(masks_list, axis=0)
boxes = np.array(boxes_list, dtype=np.float32)
labels = np.ones((len(masks_list),), dtype=np.int64)

return boxes, labels, masks

```

Listing 7: Function definition to set all seeds to ensure fair comparisons between runs.

```
def set_seed(seed=42):
```

```

    random.seed(seed)

    np.random.seed(seed)

    torch.manual_seed(seed)

    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True

    torch.backends.cudnn.benchmark = False

set_seed(42)

```

Listing 7: Function definition to set all seeds to ensure fair comparisons between runs.

```

import torch

import torch.nn.functional as F

from torchvision.ops import roi_align

from torchvision.models.detection import roi_heads

# Global variable remains the same

CURRENT_VAGUE_MASKS = None


def custom_vagueness_mask_loss(mask_logits, proposals, gt_masks, gt_labels,
                                mask_matched_idx):

    discretization_size = mask_logits.shape[-1]

    gt_mask_targets = roi_heads.project_masks_on_boxes(
        gt_masks, proposals, mask_matched_idx, discretization_size
    )

    labels = torch.cat(gt_labels, dim=0)

    indices = torch.arange(mask_logits.shape[0], device=mask_logits.device)

    pred_masks_selected = mask_logits[indices, labels]

    global CURRENT_VAGUE_MASKS

    if CURRENT_VAGUE_MASKS is not None:

        vague_crops_list = []

```



```

for i in range(len(proposals)):

    if gt_masks[i].numel() == 0:

        dummy_crop = torch.zeros((proposals[i].shape[0], 1, 28,
28), device=mask_logits.device)

        vague_crops_list.append(dummy_crop)

        continue

    h_resized, w_resized = gt_masks[i].shape[-2:]
    h_orig, w_orig = CURRENT_VAGUE_MASKS[i].shape[-2:]
    scale = h_orig / h_resized

    vague_img =
CURRENT_VAGUE_MASKS[i].unsqueeze(0).unsqueeze(0).float()

    crop = roi_align(
        vague_img,
        [proposals[i]], # box for just this image
        output_size=(discretization_size, discretization_size),
        spatial_scale=scale,
        sampling_ratio=-1
    )
    vague_crops_list.append(crop)

if len(vague_crops_list) > 0:
    vague_crops = torch.cat(vague_crops_list, dim=0).squeeze(1)
else:
    vague_crops = torch.zeros_like(pred_masks_selected)

loss = F.binary_cross_entropy_with_logits(
    pred_masks_selected, gt_mask_targets, reduction="none"
)

weights = 1.0 - (vague_crops * 0.3)

```

```
        weighted_loss = (loss * weights).mean()

        return weighted_loss

    else:

        return F.binary_cross_entropy_with_logits(
            pred_masks_selected, gt_mask_targets
        )

roi_heads.mask_rcnn_loss = custom_vagueness_mask_loss
```